现在最重要的不是继续想题目，而是把整个项目拆成一个严格分阶段推进的工程计划。你当前这篇第一作的目标已经非常明确：不是做一个泛泛的“AI优化模拟电路”系统，而是做一个面向可靠优化的模型—仿真协同框架，核心问题是代理模型在优化后期、尤其是约束边界附近会出现误判，从而导致优化震荡、仿真浪费和收敛不稳定。你后面所有工作都必须服务于这个主线。换句话说，你现在不是“边做边想”，而是要按论文叙事倒推项目建设：先把问题立住，再把系统搭起来，再把方法做深，再把实验做狠，最后把论文写出来。

第一阶段应该是项目初始化与研究协议固化阶段，这一阶段建议用一周到一周半，目标不是跑模型，而是把整个仓库变成一个“研究平台”。这一步如果做不好，后面所有东西都会乱。你要先建立仓库顶层结构，至少包括 `README.md`、`requirements.txt`、`environment.yml`、`pyproject.toml`、`configs/`、`docs/`、`scripts/`、`src/`、`data/`、`results/` 和 `tests/`。其中 `docs/` 里现在就要放进去五个文件：`proposal.md`、`methodology.md`、`simulation_pipeline.md`、`data_schema.md` 和 `experiment_protocol.md`。`proposal.md` 的功能不是介绍项目，而是明确写清楚论文问题、failure mode、方法总览、预期贡献和实验矩阵；`methodology.md` 先写旧版本方法，再预留新版本的failure-aware和utility-driven模块；`simulation_pipeline.md` 负责定义从参数到网表到仿真到结果解析的全链路；`data_schema.md` 负责规定每一条样本到底记录哪些字段；`experiment_protocol.md` 负责规定训练集、验证集、测试集的划分，所有baseline的预算、随机种子、仿真次数限制、是否共享初始样本池。这个阶段的核心贡献不是代码，而是把后面的每一步都“冻结”在可追踪协议里。你在这一阶段真正交付的成果应该是一个结构干净的仓库、第一版README、完整的docs初稿和一份明确的里程碑文档。周期建议是七天左右，最多不要超过十天，因为这一步必须快，但必须稳。

第二阶段是仿真基础设施搭建阶段，这一阶段建议两到三周，是整个项目最关键的地基。你要先选一个固定对象，强烈建议就是两级运放，不要分心去碰LDO，至少第一篇不要。然后在 `src/circuits/` 下写 `base_circuit.py`、`opamp_design_space.py`、`constraints.py` 和 `netlist_writer.py`。`base_circuit.py` 的作用是定义一个统一接口，比如 `sample_design()`、`validate_design()`、`to_netlist_params()`；`opamp_design_space.py` 里要明确列出所有可调参数，例如输入对管尺寸、镜像负载尺寸、尾电流、补偿电容、第二级尺寸等，每个参数的范围、是否取对数尺度、是否有耦合约束；`constraints.py` 里定义性能目标和约束检查器，比如 gain 最低要求、GBW 最低要求、PM 最低要求、power 最高要求；`netlist_writer.py` 负责把参数填充到 Spectre 或 SPICE 模板中。与此同时，在 `src/simulation/` 下写 `simulator_interface.py`、`spectre_runner.py`、`job_manager.py`、`result_parser.py` 和 `failure_checker.py`。`simulator_interface.py` 是统一抽象层，后面换仿真器也不会影响上层；`spectre_runner.py` 负责真正调用仿真命令；`job_manager.py` 负责批量调度和并发执行；`result_parser.py` 负责从输出日志或结果文件中抽取 gain、GBW、PM、power 等指标；`failure_checker.py` 负责区分“仿真失败”和“性能不达标”。这一步的成果不是论文方法，而是你要能从命令行一键跑出成批设计点的仿真结果。对应的脚本应包括 `scripts/generate_initial_designs.py`、`scripts/run_batch_simulations.py` 和

scripts/parse_raw_results.py。这一阶段的贡献是把模拟电路设计问题真正变成一个可编程的数据生成系统。周期建议十四到二十一天，如果这一步超过三周还没通，说明你设计空间和仿真链路太复杂了，需要立即收缩。

第三阶段是数据资产构建与边界语义标注阶段，建议一到两周。你现在不能满足于只生成 (x, y) 数据对，而是要建立真正适合“可靠优化”研究的数据格式。在 src/data/ 下写 schema.py、dataset.py、normalization.py、split.py、boundary_tagging.py 和 cache.py。schema.py 负责定义每个样本的字段，例如原始参数、归一化参数、工艺环境、性能指标、是否满足整体spec、每个约束是否满足、离最近约束边界的距离、是否仿真失败、仿真耗时、样本来源阶段、随机种子、时间戳等；dataset.py 负责读取和索引；normalization.py 负责参数和目标的标准化；split.py 负责训练集、验证集和测试集划分；boundary_tagging.py 是这一阶段新增的核心文件，它要负责根据约束计算每个样本距离边界的margin，并给样本贴上“safe / boundary / infeasible / failed”这样的标签；cache.py 负责结果缓存，避免重复仿真。你还要在 scripts/ 下补 build_dataset.py，它的功能是把初始仿真结果转成论文可用的结构化数据集。这一阶段的核心贡献是：你第一次把“边界”从一个抽象概念变成了数据字段，这为后面的failure analysis和risk modeling打下基础。周期建议七到十天，目标是得到一个至少几千条样本的干净数据集，并能输出基本统计图，例如各指标分布、约束满足率、边界样本比例。

第四阶段是基础代理模型与不确定性估计阶段，建议两周左右。先不要做任何花哨的东西，主模型就用MLP ensemble。在 src/models/ 下写 base_model.py、mlp.py、ensemble.py、mc_dropout.py 和 calibration.py。mlp.py 实现多输出回归网络，输入是设计参数，输出是多个性能指标；ensemble.py 负责训练多个初始化不同的MLP，并在预测时返回均值和方差；mc_dropout.py 作为替代不确定性实现，用于后面消融或补充证明方法不依赖具体机制；calibration.py 先预留接口，后面做真正校准时再完善。你还要写 scripts/train_surrogate.py 和 scripts/calibrate_uncertainty.py。train_surrogate.py 的功能是读取结构化数据集、训练模型、保存checkpoint、输出验证集误差；calibrate_uncertainty.py 后面会扩展成真正的不确定性校准脚本。现在这一阶段的目标不是“方法创新”，而是得到一个可用的多目标预测器，并验证它在整体分布上有基本精度。成果包括：验证集MAE / RMSE、误差分布图、不同指标上的误差统计。周期建议十到十四天。这里的贡献是：建立一个可用的 surrogate predictor，为后面证明“平均精度够但边界区仍失效”提供对照。

第五阶段是**failure mode** 识别与定量分析阶段，这是旧版升级为新版本最关键的一步，建议两周。这一阶段你不新增复杂方法，而是专门证明论文的问题真的存在。在 src/evaluation/ 下新增 failure_analysis.py、boundary_behavior.py、trajectory_analysis.py。failure_analysis.py 负责统计不同边界距离区间上的预测误差、可行性误判率和失败率；boundary_behavior.py 负责分析样本离边界越近，模型误差是否系统上升，是否存在偏向可行域或不可行域的偏移；trajectory_analysis.py 暂时可以先对随机搜索或粗糙优化轨迹做分析，后面再接优化系统。对应脚本是 scripts/analyze_failure_modes.py。你还要在 docs/ 下新增 failure_mode_analysis.md，记录分析定义、图表说明和你计划在论文里怎么用这些结论。这个阶段的交付物应该至少包括三张关键图：误差—边界距离图、误判率—边界

距离图、以及若干失败样本案例表。只有完成这一步，你的论文才真正有“问题驱动性”。周期建议十到十四天。它的贡献不是性能提升，而是把论文立论从“我觉得会失败”变成“我证明它确实在这里失败”。

第六阶段是不确定性可靠性验证与校准阶段，建议两周。这一阶段在 `src/models/` 或 `src/evaluation/` 下新增 `uncertainty_metrics.py`、`coverage_analysis.py`、`reliability_diagram.py` 和完善 `calibration.py`。`uncertainty_metrics.py` 负责计算误差与预测方差之间的相关性、rank correlation等；`coverage_analysis.py` 负责计算不同置信区间下的覆盖率；`reliability_diagram.py` 负责生成不确定性可靠性图；`calibration.py` 则负责实现简化的校准机制，例如variance scaling、isotonic-style calibration或者其他轻量后处理。对应脚本是 `scripts/evaluate_uncertainty.py`。这一步的目标是回答一个核心问题：你的不确定性到底是不是“真的反映了模型哪里容易错”。论文角度，这一阶段的贡献非常大，因为它让你后面的risk-based decision是有依据的，而不是拍脑袋。周期建议十到十四天。交付物应该包括校准前后的coverage对比、不确定性与误差相关性的图，以及一个简洁结论：经过校准后，高不确定性区域更一致地对应高误差区域。

第七阶段是旧版**UAAS**实现阶段，建议一到两周。也就是先把你旧版的 risk-aware adaptive sampling 跑通，哪怕它还偏heuristic，也必须作为过渡基线存在。在 `src/acquisition/` 下写 `base_acq.py`、`ucb.py`、`expected_improvement.py`、`risk_aware_score.py`、`boundary_aware_score.py`。在 `src/optimization/` 下写 `loop.py`、`candidate_generator.py`、`sampler.py`、`active_learner.py` 和 `trust_controller.py`。`candidate_generator.py` 负责生成候选点，可以先用随机扰动、LHS补充或简单邻域搜索；`sampler.py` 根据分数挑点；`active_learner.py` 负责把新仿真点加入数据并触发更新；`trust_controller.py` 决定高风险时是否强制仿真。对应脚本是 `scripts/run_optimization.py`。这一步的作用不是最终版本方法，而是跑出第一套完整闭环，从而验证系统能工作。周期建议七到十天。它的贡献是：你第一次把“模型预测—风险计算—仿真调用—数据回流”跑成真正系统。

第八阶段是新版本方法实现阶段：**failure-aware + utility-driven + scheduling**，这是真正冲定会的方法主体，建议三周左右。你要在 `src/acquisition/` 下新增 `feasibility_risk.py`、`misclassification_risk.py`、`utility_score.py`，并升级旧有 `risk_aware_score.py`。`feasibility_risk.py` 用来估计一个候选点满足约束的概率或风险；`misclassification_risk.py` 用来刻画“预测可行但真实不可行”的风险，这个是和failure mode直接挂钩的核心模块；`utility_score.py` 则把潜在性能收益、可行性概率、误判风险和不确定性合在一起，形成真正的task-aware采样指标。与此同时，在 `src/optimization/` 下新增 `scheduler.py` 和 `decision_logger.py`。`scheduler.py` 负责决定当前候选点是直接采用代理预测、进入局部模型驱动搜索，还是必须调用真实仿真；`decision_logger.py` 负责把每一轮决策完整记录下来，包括均值、方差、边界margin、risk score、utility score 和最终动作。你还要在 `configs/optimizer/` 下新增 `uaas_plus.yaml`，把旧版 `uaas.yaml` 和新版分开。对应文档上，在 `docs/` 下新增 `uncertainty_validation.md` 和升级 `methodology.md`。这

一阶段的贡献是把旧版heuristic系统升级为真正面向failure mode的决策框架。周期建议十四到二十一天。完成后，你的系统应该已经能支撑论文主方法。

第九阶段是**baseline**与实验协议完全固化阶段，建议一周。这里重点不是发明新方法，而是确保所有对比都公平、可复现、能支撑论文结论。你要在 `configs/optimizer/` 下配置好 `random_search.yaml`、`bo_baseline.yaml`、`surrogate_only.yaml`、`uaas_old.yaml` 和 `uaas_plus.yaml`。在 `src/evaluation/` 下写 `benchmark.py`、`ablation.py`、`metrics.py` 和 `visualization.py`。`benchmark.py` 负责批量跑不同方法；`ablation.py` 负责去掉校准、去掉utility、去掉misclassification risk、关闭在线更新等模块做消融；`metrics.py` 负责统一计算仿真次数、成功率、最终性能、后期震荡度、误判率、wall-clock等指标；`visualization.py` 则负责统一输出论文图表。脚本上要有 `scripts/evaluate_baselines.py` 和 `scripts/plot_main_results.py`。这一阶段的贡献是把实验从“能跑”变成“有协议、有基准、有可比性”。周期建议五到七天。

第十阶段是困难场景与杀手级实验阶段，建议两周。你现在最需要的是让baseline在关键场景下明显失败，所以这一阶段非常重要。在 `src/circuits/` 下新增 `hard_cases.py`，里面显式定义若干困难任务，例如极限phase margin要求、低功耗 + 高GBW冲突、多约束同时收紧等；在 `configs/circuit/` 下新增 `opamp_hardcase_tightpm.yaml`、`opamp_hardcase_conflict.yaml` 等；在 `docs/` 下新增 `hard_case_protocol.md`，写清楚这些困难场景为什么有代表性、各自针对什么failure mode；脚本上写 `scripts/run_hardcase_benchmarks.py`。这一步的目标不是多做实验，而是做对实验。真正有价值的结果不是“平均上提升5%”，而是在某些关键约束下传统方法后期频繁误判、反复仿真，而你的方法稳定收敛。这一阶段周期建议十到十四天。它的贡献是论文的“杀手图”和“杀手表”。

第十一阶段是整体成本、**OOD**与扩展实验阶段，建议一周到十天。你需要补全论文完整性，在 `configs/experiment/` 下加入 `exp_ood.yaml` 和 `exp_wallclock.yaml`；在 `scripts/` 下写 `evaluate_wallclock.py`；在 `src/evaluation/` 下完善 `ood.py`。OOD不必做得太重，可以只是改变负载、电源、或spec组合，验证方法在轻微分布变化下是否仍有效。wall-clock实验则要统计仿真时间、训练时间、推理时间、总时间。你还要在 `docs/` 下新增 `wall_clock_analysis.md`。这一阶段的贡献是回答EDA领域很现实的问题：不仅仿真次数变少了，实际总时间是否也变少了；不仅在单一设定下工作，在轻量变化下是否依然成立。周期建议七到十天。

第十二阶段是论文图、文档和写作绑定阶段，建议两周。到这一步，你要把工程资产转化成论文资产。在 `docs/` 下新增或完善 `paper_figures_plan.md`，逐张列出论文图：系统框架图、failure mode图、不确定性校准图、主结果收敛图、困难场景对比图、决策分布图、wall-clock图。你还要把 `README.md` 升级成论文压缩版，把 `proposal.md` 改成接近 introduction + contributions 的版本，把 `experiment_protocol.md` 写得和真正论文supplementary一致。脚本上必须保证 `scripts/reproduce_paper.sh` 能串起来主要实验和画图流程。这一阶段的贡献不是新结果，而是让整个仓库达到“审稿人或导师看了会相信你能复现论文”的程度。周期建议十到十四天。

如果按一个相对稳妥的节奏来排，总项目周期大约是三个月到四个半月。第一月应该完成项目初始化、仿真基础设施、数据集构建和基础模型；第二月完成failure mode分析、不确定性校准和旧版闭环；第三月完成新方法、baseline和困难场景实验；第四月前半完成扩展实验、图表和写作绑定。你完全可以把里程碑设成这样：第一个月结束时必须有稳定数据集和一个可用surrogate；第二个月结束时必须有failure mode图和uncertainty validation图；第三个月结束时必须有主结果和困难场景结果；第四个月只允许做修补和写作，不允许再大改系统主干。

从项目管理角度看，你现在每一阶段都要产出三样东西，而不是只产出代码。第一样是“可运行模块”，也就是对应的 `.py` 文件和脚本；第二样是“实验资产”，也就是表格、图、日志和配置；第三样是“文档资产”，也就是 `docs/` 里的说明和你对这一阶段贡献的总结。只有这三样同步积累，最后你写论文才不会空。你这篇第一作的最终贡献应该被稳定地组织成四条：一是系统性分析并量化了代理模型在模拟电路边界区域的失效模式；二是提出了经过校准的不确定性感知、面向误判风险的决策机制；三是提出了utility-driven主动采样与仿真调度闭环；四是在典型模拟电路及困难场景下证明了该方法在仿真预算、后期稳定性和整体效率上的优势。项目的每个阶段都必须反过来服务这四条，不相关的东西一律不要扩。